

Yellowbrick Architecture

High Level Architecture Review

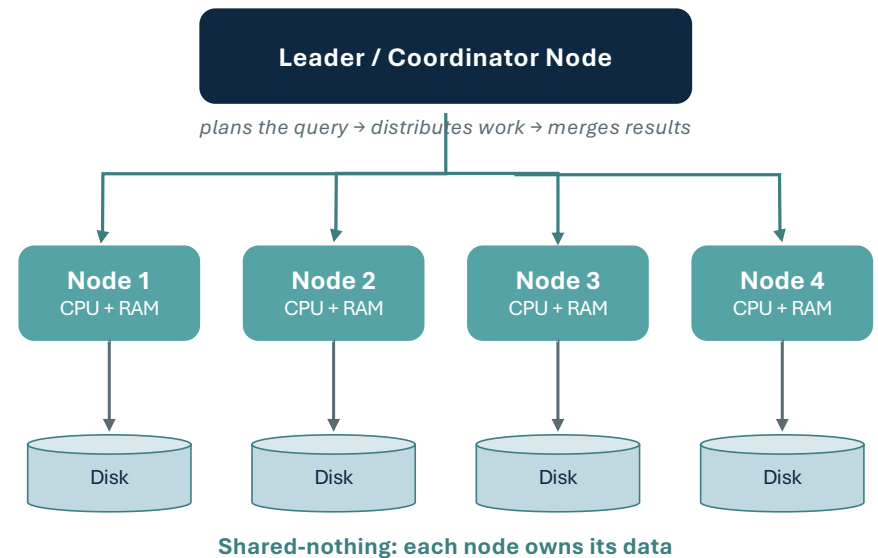
Massively Parallel Processing (MPP) Databases

What is MPP?

Work is split across many independent nodes that each process their own slice of the data in parallel, then combine results. A “shared-nothing” design: each node has its own CPU, memory, and storage.

Key Advantages

- **Linear scalability** Add nodes to grow capacity and speed near-proportionally.
- **Massive parallelism** Every node works on the query at the same time.
- **No shared bottleneck** Independent CPU, memory & disk avoid contention.
- **Built for analytics** Excels at large scans, joins and aggregations.



Yellowbrick | Tasks | Health | Support | Hi michael.suarez@yellowbrick.com

INSTANCE MANAGEMENT > YBDEMO > CLUSTERS

defcluster

Privileges | Actions | **Cluster Running** | Last 5 Minutes

Summary | Statistics | Logs

Active Sessions
3

Total Local Storage
4.0 TB

Cache Efficiency
11.43%

Total Read I/O
117.4 MB

Total Write I/O
0 bytes

Overview

16 vCPUs

1 Node Count

State Running
Auto Suspend 1800s
Auto Resume true
WLM Profile default

Hardware Instance Type
small-v2 | 16 vCPUs | 116 GB RAM | 4 TB Local Storage

Workload Management
Resource Pool Memory | Active Queries

PU/L	%MEM/R	%MEM/L	IP
n/a	n/a	n/a	10.10.83.108
0	5	1	10.10.46.4
n/a	100	n/a	10.10.98.159
n/a	24	n/a	10.10.68.9
0	n/a	18	10.10.30.215
n/a	n/a	n/a	10.10.24.128

Demo

- One Node Running
- One Worker Pod in K8s Cluster
- Start Query, Watch Timer
- Table Structure and Size
- 16.4 Seconds to run query
- Increase Cluster Size to 4
- Scriptable SQL Statement
- Watch K8s Deployment scale
- Rerun Query
- **4.1 Seconds, 4x Faster**
Linear Scalability
- Scale down using SQL Command
- K8s Pods terminating

MPP Key Concepts

How data lands on the nodes drives everything — these are the design and performance levers.

1

Data Distribution

How rows are spread across nodes: hash on a key, replicate small tables, or round-robin. The foundational table-design choice.

2

Data Skew

Lopsided key values overload some nodes. A query is only as fast as its slowest node, so an even spread is essential.

3

Join Co-location

Tables distributed on the same join key are joined locally on each node — no network traffic. The single biggest performance win.

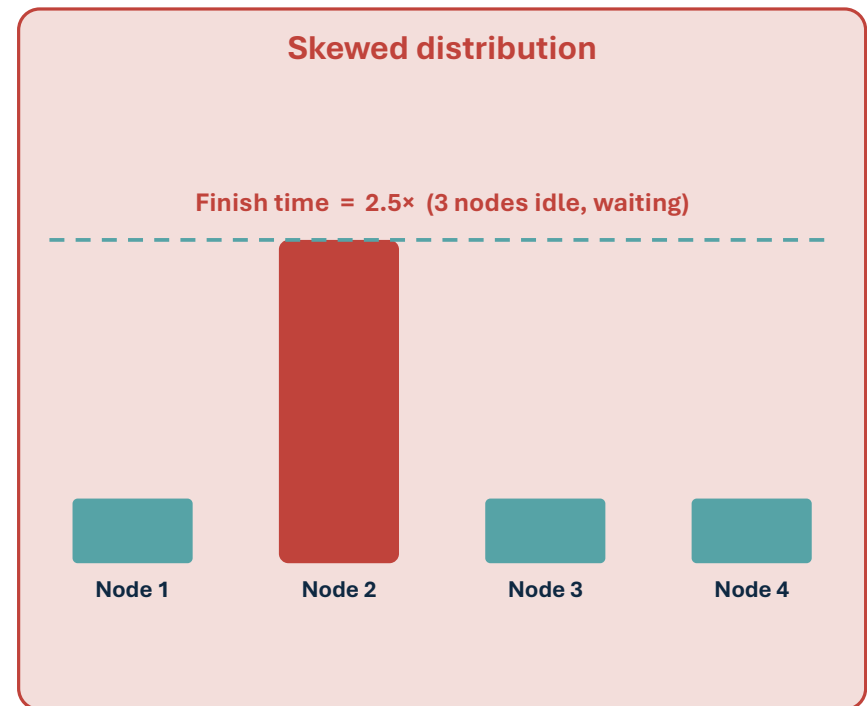
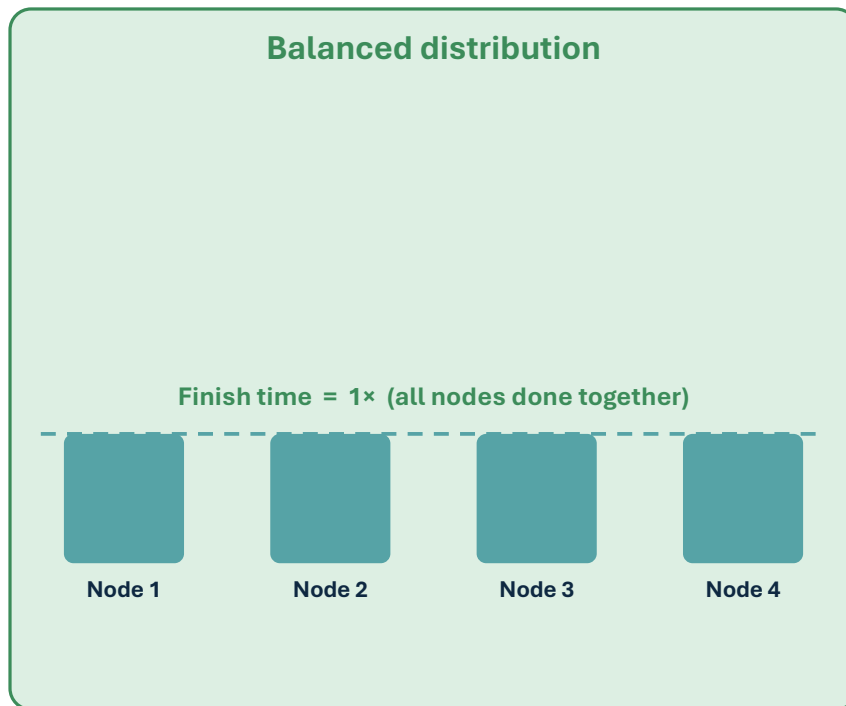
4

Data Movement

When keys don't align, rows are redistributed or a table is broadcast to every node. Correct but costly — spot it in query plans.

Data Skew: A Query is Only as Fast as Its Slowest Node

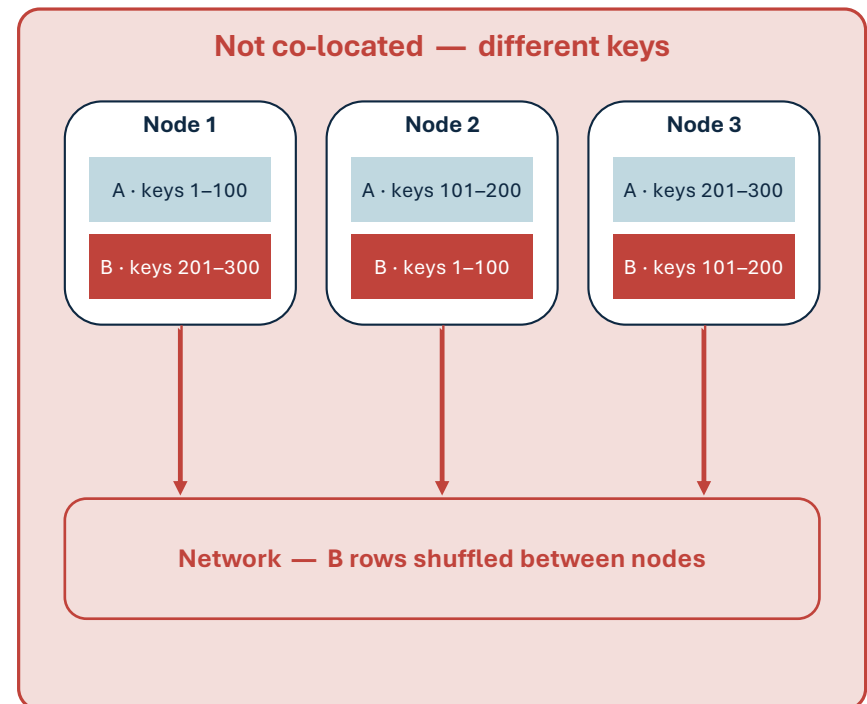
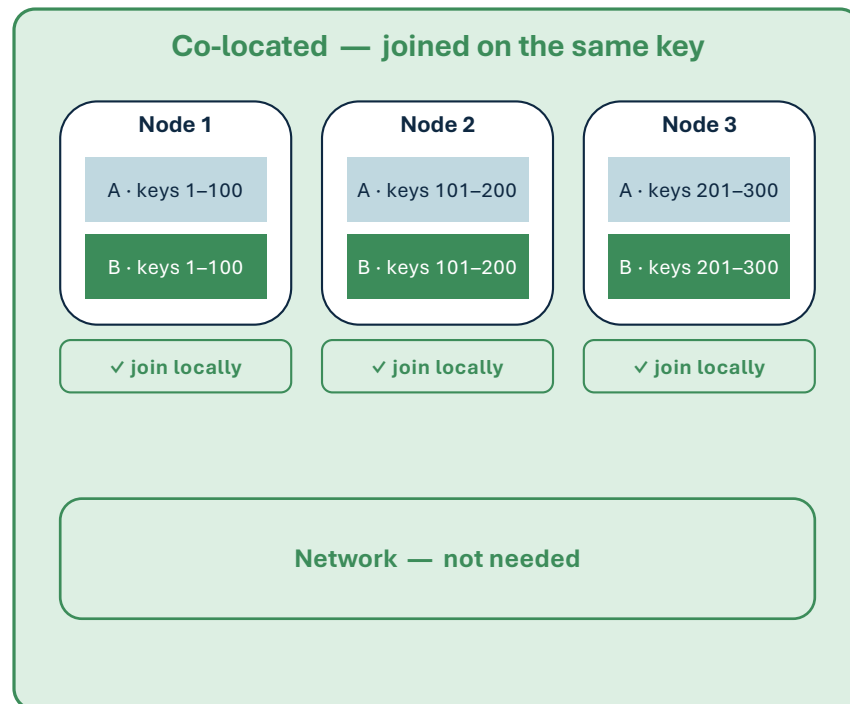
Same total data, four nodes — but a query finishes only when its last node finishes.



Takeaway: pick a distribution key with many evenly-spread values — skew wastes the parallelism you paid for.

Join Co-location: Avoid the Network Shuffle

Joining A to B on the distribution key keeps every match on one node — otherwise rows must cross the network.



Takeaway: distribute tables you join often on the same key — co-located joins skip the costly network shuffle.

Columnar Data Storage

Row vs. Column orientation

Traditional databases store a whole row together. Columnar databases store each column together. Analytic queries usually touch a few columns across many rows — so reading by column moves far less data.

Key Advantages

- **Reads only needed columns** Skip columns the query never references.
- **High compression** Similar values stored together compress far better.
- **Vectorized processing** Operate on column batches for CPU efficiency.
- **Less I/O, faster scans** Ideal for aggregations over huge tables.

Row store — row values kept together

ID	Name	Region	Sales

reads the whole row even for 1 column

Column store — each column kept together

ID	Name	Region	Sales

scan just the columns you need

Columnar Key Concepts

Columnar storage is about reading as little data as possible — these concepts make that happen.

1

Column Projection

Each column is stored separately, so only referenced columns are read. Narrow column lists beat SELECT *.

2

Sort Order / Clustering

Physically ordering data (e.g., by date) lets the engine skip the blocks a query doesn't need.

3

Zone Maps / Block Pruning

Each block stores its min and max. Blocks outside a filter's range are skipped without being read — pairs with sort order.

4

Compression / Encoding

Storing like values together (dictionary, run-length, delta) compresses far better than rows, cutting I/O.

5

Vectorized Execution

The engine processes columns in batches rather than row-by-row, maximizing CPU efficiency on scans and aggregations.

Block Pruning: Skip What You Don't Need

The table is sorted by `sale_date`. Every block stores its min and max — so blocks that can't match the filter are never read.

```
SELECT SUM(amount)
FROM   sales
WHERE  sale_date BETWEEN
       '2026-03-01'
       AND '2026-03-31';
```

2 of 8

blocks read

75% of I/O avoided

Storage blocks of `sales` — physically sorted by `sale_date`

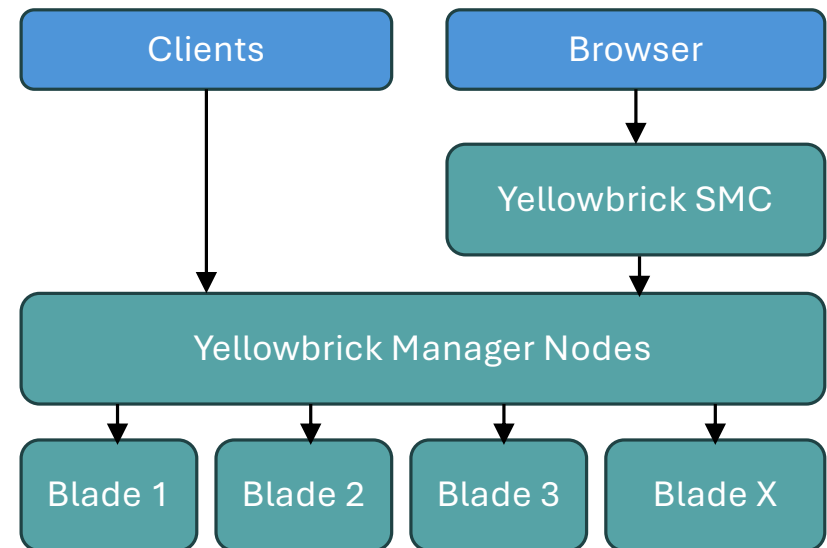
Block 1	Jan 01 – Jan 24	skipped
Block 2	Jan 25 – Feb 16	skipped
Block 3	Feb 17 – Mar 11	✓ READ
Block 4	Mar 12 – Apr 04	✓ READ
Block 5	Apr 05 – Apr 28	skipped
Block 6	Apr 29 – May 22	skipped
Block 7	May 23 – Jun 15	skipped
Block 8	Jun 16 – Jul 09	skipped

Demos

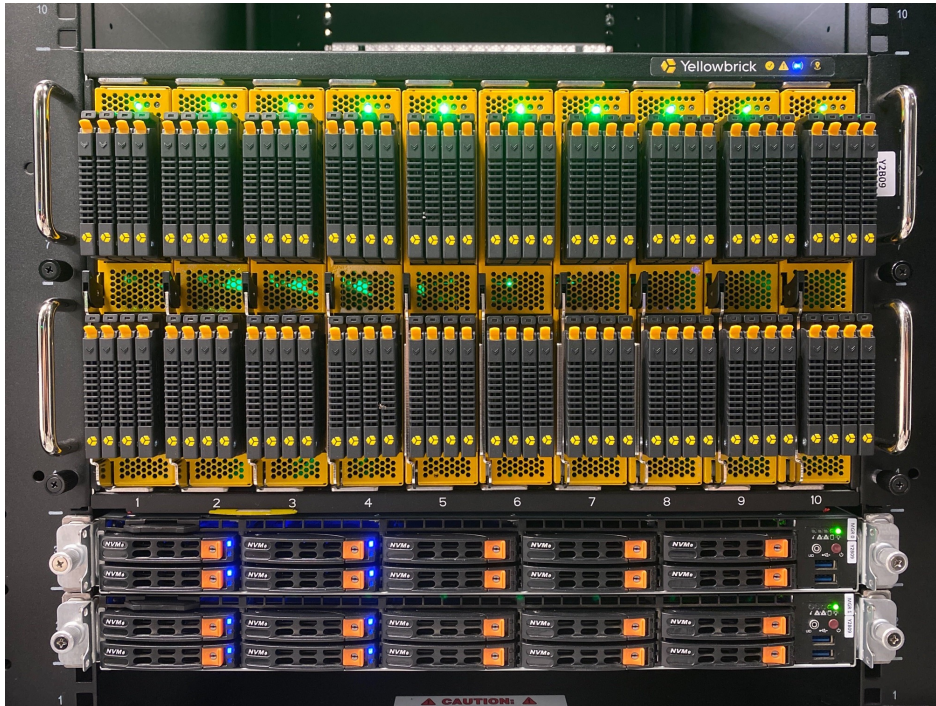
- See effects of
 - Data Skipping
 - Skew
 - Colocation

Yellowbrick Components: On Premise

Layer	Components
Client	App servers, client laptops, etc.. Tools, drivers, browsers, bulk services
Manager	Yellowbrick SMC GUI web based management interface
Front End	Manager Nodes Metadata & Row Data Storage
Backend	Blades SQL Processing CPU, Persistent Data
Object Storage	N/A



Various Yellowbrick Configurations

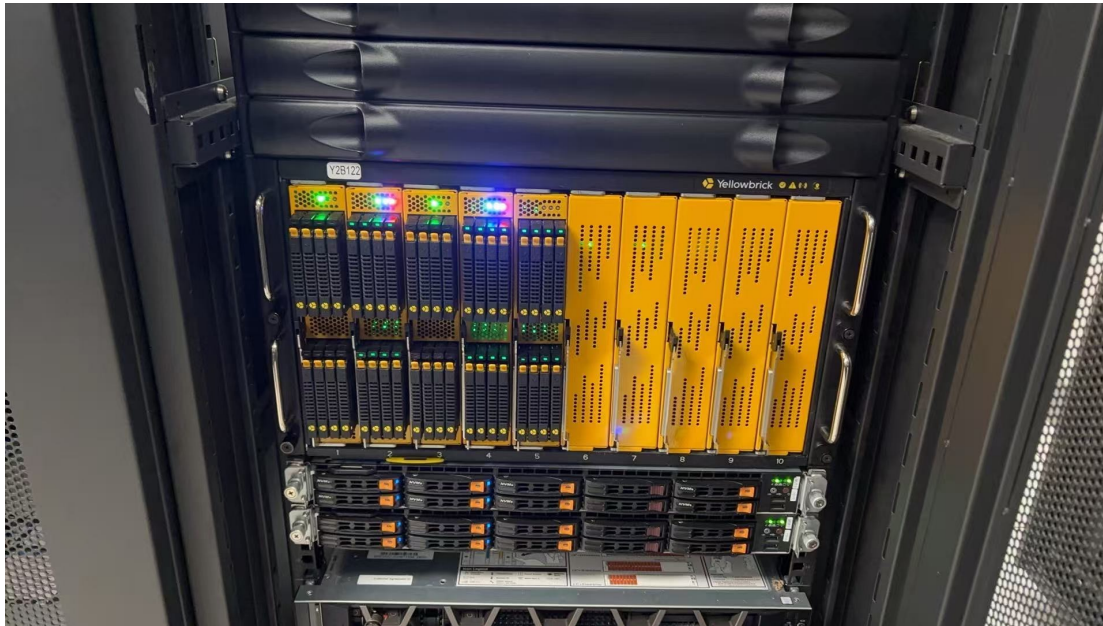


A Full 10 Blade Chassis

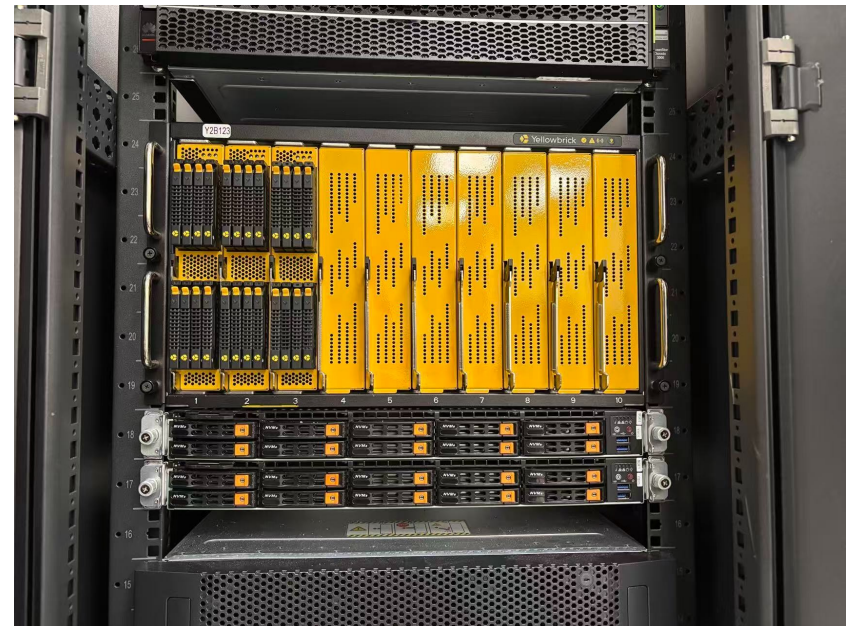


80 Blades – Yellowbricks largest deployment

Actual Melco Hardware



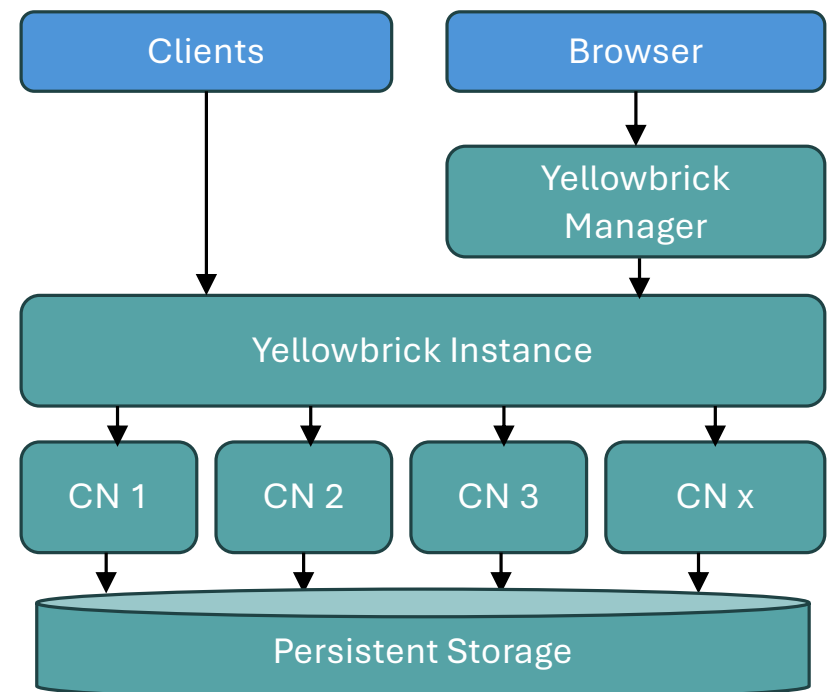
YB122 - Production



YB123 – Dev & DR

Yellowbrick Components: Cloud

Layer	Compoents
Client	App servers, client laptops, etc.. Tools, drivers, browsers, bulk services
Manager	Yellowbrick Manager GUI web based management interface
Front End	Instance Metadata & Row Data Storage
Backend	Compute Nodes SQL Processing CPU, Hydrated Data
Object Storage	Unlimited Persistent Data Storage for Cloud S3 Compatible Interface to Object Storage



Cloud Key Concepts

What makes cloud deployment different from on-premise

1

Separation of Compute & Storage

Data lives permanently in object storage (S3). Compute nodes are independent — scale each up or down without moving data.

2

Hydrating Compute Nodes

Before a query runs, data is pulled from object storage to compute node local SSDs. Already-warm data skips this step — hydration is the cold-start cost.

3

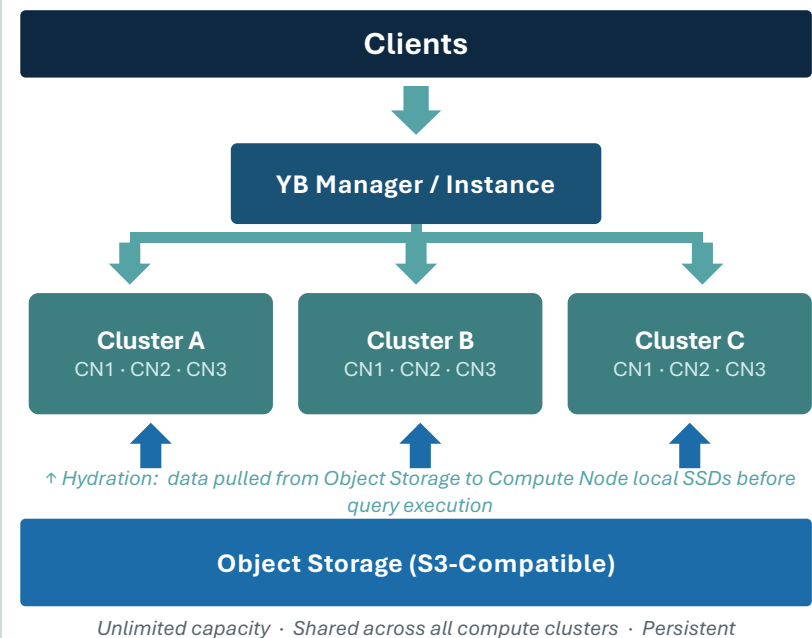
Multiple Compute Clusters

Multiple independent clusters share the same object storage simultaneously — enabling workload isolation, multi-tenancy, and different resource tiers per team.

4

Elastic Scaling

Add or remove compute nodes on demand. Pause, resize, or spin up a new cluster without touching the data in object storage.



Query Execution Lifecycle

From SQL submission to result — what happens inside Yellowbrick

1

Parse & Plan

SQL arrives at the Manager's PostgreSQL interface. The planner builds an execution plan — choosing join strategies, distribution, and flagging any required data redistribution.

2

Generate & Cache C++ Code

The plan compiles to native C++ code and is distributed to compute nodes. Code is cached by plan_id — identical plans on repeat runs skip recompilation entirely.

3

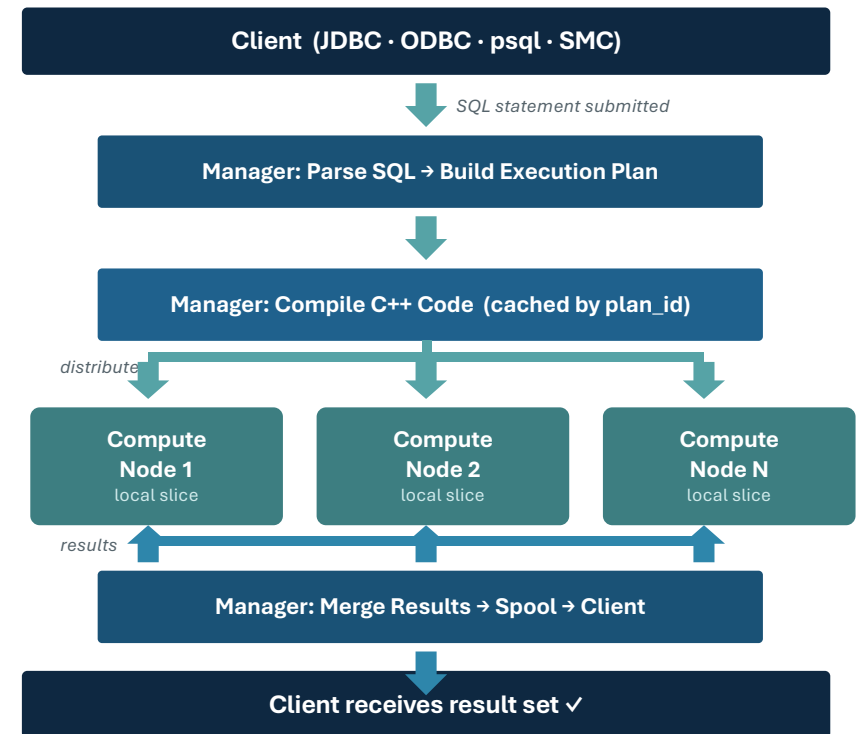
Parallel Execution

All compute nodes execute simultaneously on their local data slice. Range-restricted scanning skips irrelevant blocks; redistribution only occurs when co-location requires it.

4

Merge & Return to Client

Each node streams partial results to the Manager, which merges them, spools to its local drive, and returns the final result set to the client.



Thank You